

EVALUACIÓN GLOBALIZADORA A – TÉCNICAS DIGITALES III

CONTROL DE ILUMINACIÓN

Eirea, Alejandro
e-mail: ale.impa97@gmail.com
Mehle, Ignacio
e-mail: igmehlegtr@gmail.com

Contenido

RESUMEN:	1
1 INTRODUCCIÓN Y FUNDAMENTACIÓN	1
2 DIAGRAMA EN BLOQUES	2
3 ALCANCE Y FUNDAMENTACIÓN	2
4 HARDWARE	2
4.1 MICROCONTROLADOR	2
4.2 LUXÓMETRO BH1750	2
4.3 EEPROM Y RTC	2
4.4 DISPLAY	2
4.5 ENCODER	2
4.6 SALIDA PWM	3
5 PCB Y ARMADO	3
6 CONCLUSIONES	3
7 REFERENCIAS	4

RESUMEN:

El sistema debe controlar el nivel de iluminación en un ambiente, el cual debe poder setearse previamente, variando el brillo de tiras LED mediante PWM. El nivel de luz debe mantenerse estable en el nivel seteado frente a cambios en la iluminación. Se contempla la posibilidad de que el nivel puede setearse de forma remota mediante una aplicación Android comunicada por bluetooth. Los seteos de iluminación en el tiempo se pueden almacenar en un log de eventos. Todo el sistema se basa en una placa de desarrollo Raspberry Pi Pico 2, utilizando el sistema FreeRTOS.

1 INTRODUCCIÓN Y FUNDAMENTACIÓN

Se plantea un micro ambiente de prueba en forma de una pequeña caja, con una fuente de luz que consiste en tiras LED, las cuales se ajustan mediante PWM. La caja debe poder abrirse parcialmente, para evaluar la respuesta del sistema a cambios en la iluminación.

El sistema se fundamenta en una placa de desarrollo Raspberry Pi Pico 2, la cual posee un doble núcleo Cortex M33 de 32 bits, y permite la ejecución del sistema operativo de tiempo real FreeRTOS, el cual comanda la ejecución del programa atendiendo a los eventos de control y operación de forma más eficiente y responsiva frente a una solución en *baremetal*.

El nivel de luz es medido con un luxómetro digital BH1750. Se emplea este módulo porque nos permite obtener una medición directa ya procesada del nivel de iluminación frente a la medición analógica con LDR los cuales necesitan un contraste para trazar la curva de respuesta en función de la iluminación.

Se busca poder setear el nivel de iluminación, y que el sistema reaccione ajustando el brillo de la tira LED para compensar la iluminación natural del ambiente y así lograr la iluminación buscada.

Para llevar un registro de los seteos se recurre a un RTC DS3231, y a una memoria EEPROM AT24C32. De esta forma buscamos almacenar en forma de log de eventos los últimos seteos y el momento en que fueron guardados. Se decide emplear la memoria EEPROM porque con un tamaño de 32Kbits nos permite almacenar más de 200 entradas en el log, algo suficiente para el alcance del proyecto. Por otro lado, se aprovecha el RTC presente junto al módulo de memoria, para el cual ya se contaba con una librería implementada por un proyecto anterior.

Para visualización y control del sistema utilizamos de forma local un display LCD controlado por bus I2C, y un encoder con pulsador para el seteo de niveles. De esta manera tenemos una solución ágil y minimalista para el usuario. En el menú se permite setear:

- NIVEL DE LUX
- ALARMA MAX
- ALARMA MIN
- CURVA DE AJUSTE
- CALIBRACIÓN

Las alarmas representan 2 niveles de iluminación frente a los cuales el sistema ya no puede compensar. Ambos pueden ser fijados por el usuario dentro de los rangos máximos presentes en la calibración. Para esto se implementan dos leds representando las alarmas.

La curva de ajuste representa la posibilidad de ajustar la iluminación de forma directa o de forma gradual en pasos.

La opción de calibración permite llamar a la función de calibración, que permite ajustar el control PID de la salida PWM. Las variables de calibración se almacenan en los primeros bytes de la memoria EEPROM.

Una UART y un puerto I2C deben ser reservados en el pinout de la Raspberry Pi Pico 2 para su posterior uso como módulo esclavo de una Raspberry Pi 4.

2 DIAGRAMA EN BLOQUES

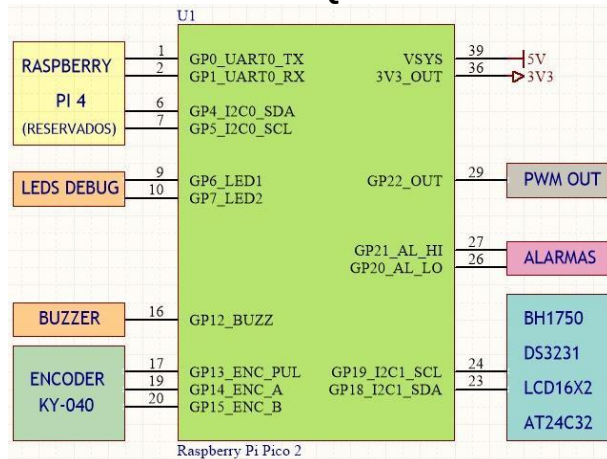


Fig. 1 Diagrama en bloques

3 ALCANCE Y FUNDAMENTACIÓN

El alcance del proyecto fue definido como un control de iluminación a una tira led con un error menor a 5 lux. El hardware usado como luxómetro, el módulo BH1750 se eligió porque entrega el valor directamente sin necesidad de contraste y por la simplicidad de lectura al manejarse por bus I2C. Luego el proyecto indicaba la necesidad de crear un log de eventos y configuración con lo cual se eligió el módulo de DS3231 que es un RTC en tiempo real que además viene con una memoria eeprom AT24C32. De esta manera con el mismo módulo se resolvía el tema del control de tiempo para los eventos y la plataforma para guardarlos.

Se eligió un encoder para navegar por el menú y una lcd para mostrar este y las mediciones. Como plataforma para desarrollar el firmware necesario para el proyecto se usa la placa de desarrollo Raspberry Pi Pico 2, que es una de las que se trabaja el presente año en la cátedra.

Desde el punto de vista del software, se tienen 5 tareas que se ocupan de los periféricos utilizados. Una sola se activa por interrupción: la tarea que maneja los botones del encoder, y después son 2 tareas más: Una que configura la eeprom que es donde se guardan los seteos del usuario, y una para el control PID propiamente dicho. La comunicación entre tareas se hace a través de semáforos y los datos se comunican por medio de colas.

4 HARDWARE

4.1 MICROCONTROLADOR

Se emplea la placa de desarrollo Raspberry Pi Pico 2, la cual permite correr FreeRTOS como base del sistema. La placa cuenta con un microcontrolador RP2350 de dos núcleos Cortex M33 @ 150 MHz, unidad de punto flotante, 520 kB de SRAM, 2 MB de memoria flash, y utilizamos la extensión Raspberry Pi Pico de Visual Studio Code como IDE.

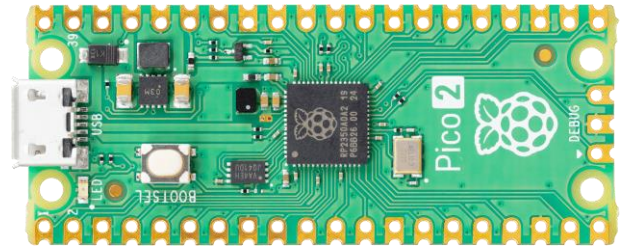


Fig. 2 Raspberry Pi Pico 2

4.2 LUXOMETRO BH1750

El módulo BH1750FVI es un luxómetro digital con interfaz I2C. Se alimenta con 3,3V y entrega una lectura directa de la iluminación ambiente, con un rango de 1 a 65535 lx. Tiene una resolución de 1 lx en modo High-Res (tiempo de medida de 120 ms), y de 4lx en modo Low-Res (tiempo de medida de 16ms).

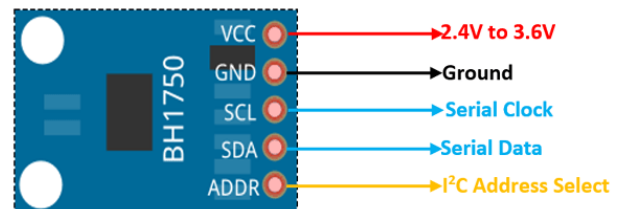


Fig. 3 Luxómetro digital BH1750

4.3 EEPROM Y RTC

Utilizamos un módulo que posee una EEPROM AT24C32 de 32kbits y un RTC DS3231 el cual es una actualización del más difundido DS1077, ya que puede alimentarse con 3,3V.

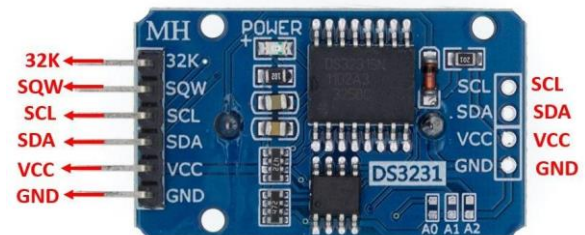


Fig. 4 EEPROM AT24C32 y RTC DS3231

4.4 DISPLAY

Se emplea un display LCD de 16x2 que emplea el controlador HD44780, y junto a él utilizamos un expansor I2C a 8 bits serie/paralelo PCF8754, como interfaz I2C del LCD. Como el LCD se alimenta a 5V, y nuestro bus está referenciado a los 3,3V que utiliza la Pico 2 y los demás módulos, agregamos un convertor de niveles para poder utilizar el mismo bus.



Fig. 5 Display LCD 16X2

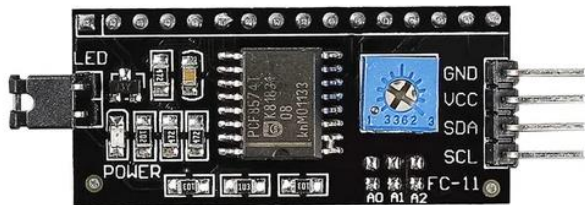


Fig. 6 Conversor I2C-Paralelo PCF8754

4.5 ENCODER

Para control por parte del usuario se empleó un encoder rotativo de 20 pasos KY-040, el cual posee un pulsador incorporado. Para facilitar la lectura de las transiciones, empleamos antirrebote físico con filtro RC y compuertas Schmitt Trigger para digitalizar los flancos y reducir la lógica de lectura.

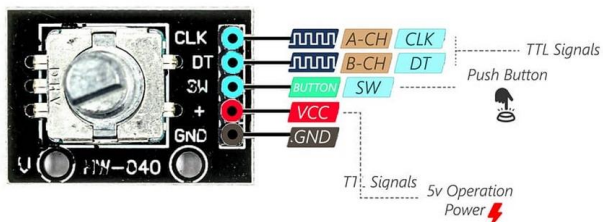


Fig. 7 Encoder KY-040

Estas entradas se leen con interrupciones por hardware, que disparan tareas controladoras de FreeRTOS las cuales analizan la lógica del paso, si es horario o antihorario.

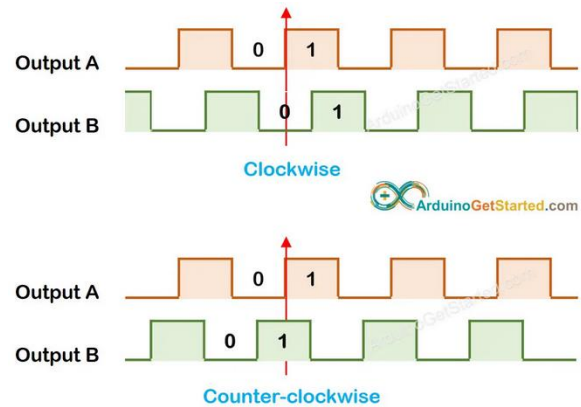


Fig. 8 Transiciones del encoder

4.6 SALIDA PWM

El control del brillo de la tira led se efectúa por medio de una etapa de potencia MOSFET en corte saturación, usando un IRFZ44N de bajo $R_{DS(on)}$ y un driver tótem pole. Se agrega un buffer con filtro RC para medición del valor medio del PWM como testpoint.

5 PCB Y ARMADO

El PCB se diseñó con el software Kicad Versión 8. Se colocaron testpoints en:

- TP1: PWM LEVEL
- TP2: PWM OUT
- TP3,4: GND

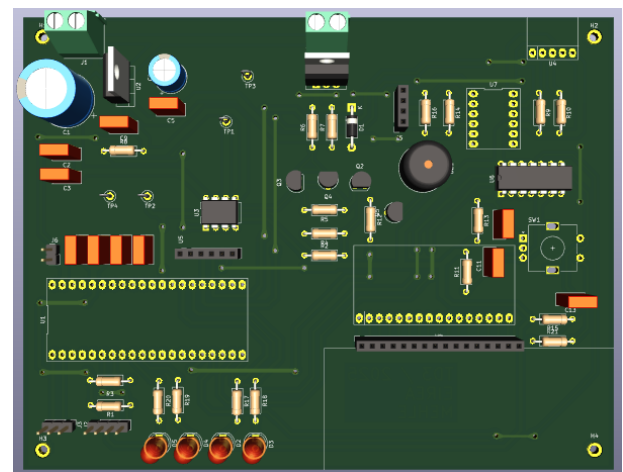


Fig. 9 Frente del PCB

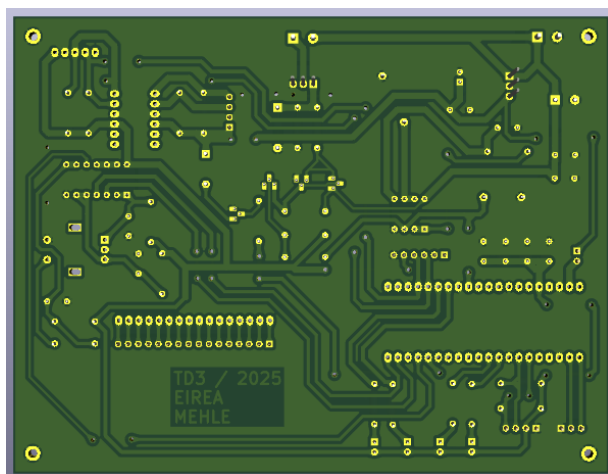


Fig. 10 Reverso del PCB

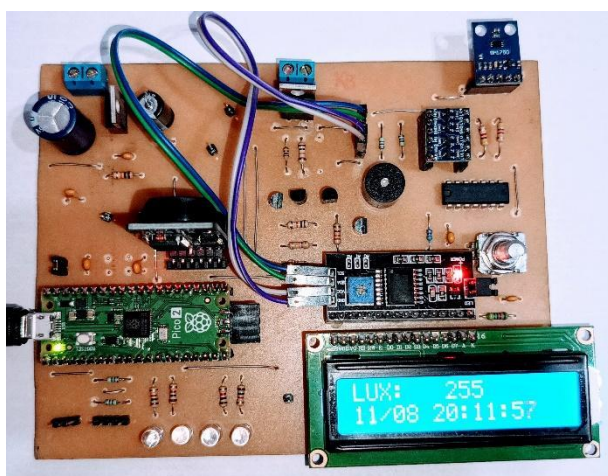


Fig. 11 PCB ensamblado

6 CONCLUSIONES

RESULTADOS OBTENIDOS:

Se pudo lograr un sistema que ajusta el brillo de la luminaria (leds) adecuándose a la iluminación requerida por el usuario dentro de los niveles de ajuste que brinda el microcontrolador y la luminaria dada.

Se pudo lograr que el sistema guarde los seteos en la memoria EEPROM e implementar la funcionalidad de leer los settings guardados en el tiempo y borrarlos, lo que será utilizado más adelante en la EGB.

Se logró que el sistema permita correctamente configurar un setpoint de nivel de lux por parte del usuario, y que se puedan configurar dos tipos de ajuste según variaciones en la medición ambiente. Cada nuevo setting es guardado con fecha en memoria no volátil.

Se logró que el nivel pueda ajustarse por debajo de la banda de error del luxómetro gracias al filtrado de muestras y la acción del control PI, llegando a un ajuste en régimen permanente por debajo de los 5 lux de error absoluto con respecto al setpoint.

Cabe destacar que la capacidad de ajuste del sistema depende del ambiente y de la luminaria utilizada, en este caso el sistema está planteado a modo demostrativo, pensado para hacer un dimerizado de tiras LED de 12V.

La principal conclusión es que si la respuesta de la planta es demasiado rápida, en este caso la respuesta de los leds al ciclo de actividad del PWM, a la velocidad de medición, se hace muy difícil el control usando la aplicación de PID.

Para resolver este problema lo que hicimos fue aplicar un filtro EMA: **Exponential Moving Average**, que lo que hace es ralentizar la respuesta de la planta y además tuvimos que cambiar la manera de medición de forma continua a por disparo cada 25 ms para que el control fuera posible.

De forma empírica pudimos llegar a 2 ajustes de respuesta de la planta, uno rápido y uno lento, que dependen del filtrado aplicado al PWM. Empíricamente pudimos llegar a dos constantes para cada planta, utilizando un control PI en cada caso. Se observó que para la respuesta rápida la adición de la acción integral no arrojaba mejoras sustanciales y sumaba sobrepicos no deseados al ajuste.

PROBLEMAS ENFRENTADOS:

La librería de la eeprom at24c32 fue realizada de cero según los diagramas I2C del datasheet. Tuvimos que resolver dos problemas en el código, un bloqueo de escritura, al tener un retardo de tiempo (función sleep_ms()) que entraba en conflicto con el scheduling del RTOS, y un error de tipeo en la función de lectura el cual direccionaba mal el address. Esto se podría haber solventado utilizando librerías de terceros o bien con un mejor testeo de las funciones, bajo RTOS, no solo en baremetal.

La presencia de dos niveles lógicos para distintos módulos es una complejidad a la hora del conexionado y testeo (uno de los luxómetros se dañó y conseguir un reemplazo implicó un retardo en el curso del desarrollo). El armado de la caja de prueba no obtuvo los resultados que esperábamos a la hora de provocar una variación controlable en el nivel de luz que incide sobre el luxómetro.

Para implementar las funciones de log de settings y borrado de memoria previstas para la EGB, se planteó una tarea que realizaba dichas funciones de acuerdo a comandos ingresados por consola sobre USB. Esta tarea terminaba bloqueando a la tarea de control, interfiriendo por completo con el algoritmo de control. La solución de compromiso fue eliminar esta tarea y adosar las funciones al menú de opciones, contemplando que se pueda probar estas funcionalidades sin interferencias y queden previstas para ser manejadas de forma externa por otras interfaces como I2C o UART.

Por otro lado, el algoritmo de autotune del control PID no arroja resultados concluyentes. Si bien la función está presente en el programa, en este punto no modifica las constantes del control hasta que se pueda llegar a una versión funcional en el futuro, trabajando el control con constantes empíricas predefinidas.

Quedan por resolver cuestiones menores en la lectura de los pasos del encoder, optimizar el ajuste de los

parámetros en el menú y ajustar los punteros en las funciones de log de settings y borrado de memoria, así como una indicación de memoria llena.

7 REFERENCIAS

[0] Apuntes de Cátedra de Electrónica Industrial
<https://leifrautn.blogspot.com/search/label/Electr%C3%B3nica%20industrial%201>

[1] FreeRTOS
<https://www.freertos.org/>

[2] Raspberry Pi Pico 2:
<https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf>

[3] BH1750:
<https://www.alldatasheet.es/datasheet-pdf/view/338083/ROHM/BH1750FVI.html>

[4] DS3231:
<https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>

[5] AT24C32:
<https://www1.microchip.com/downloads/en/DeviceDoc/doc0336.pdf>

[6] KY 040:
<https://eloctavobit.com/modulos-sensores/codificador-rotatorio-ky-040-rotary-encoder>

ANEXO 1 – GUÍA DE CÓDIGO

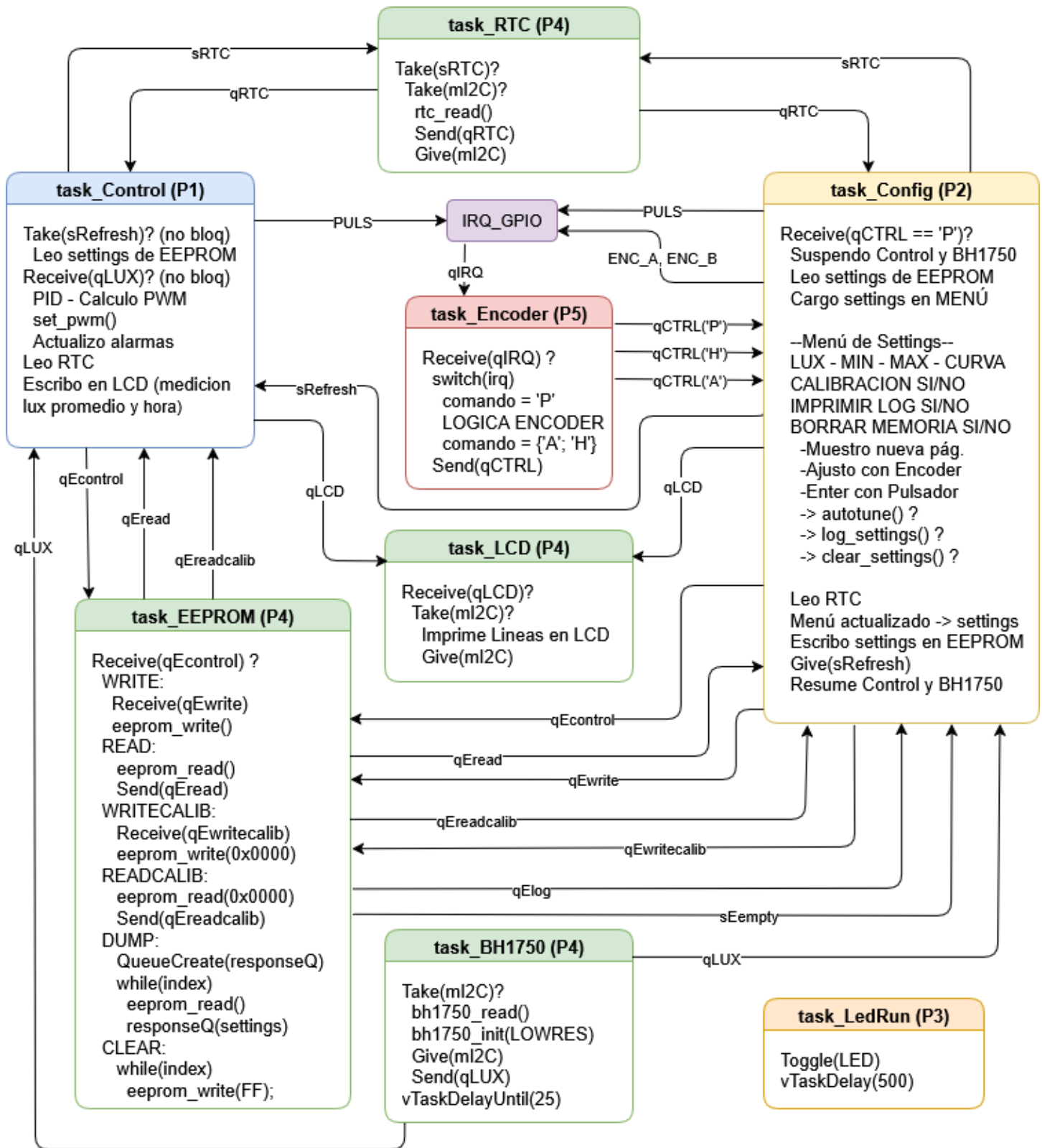


Fig. 12 Guía de tareas de FreeRTOS

ANEXO 2 – ESQUEMÁTICO

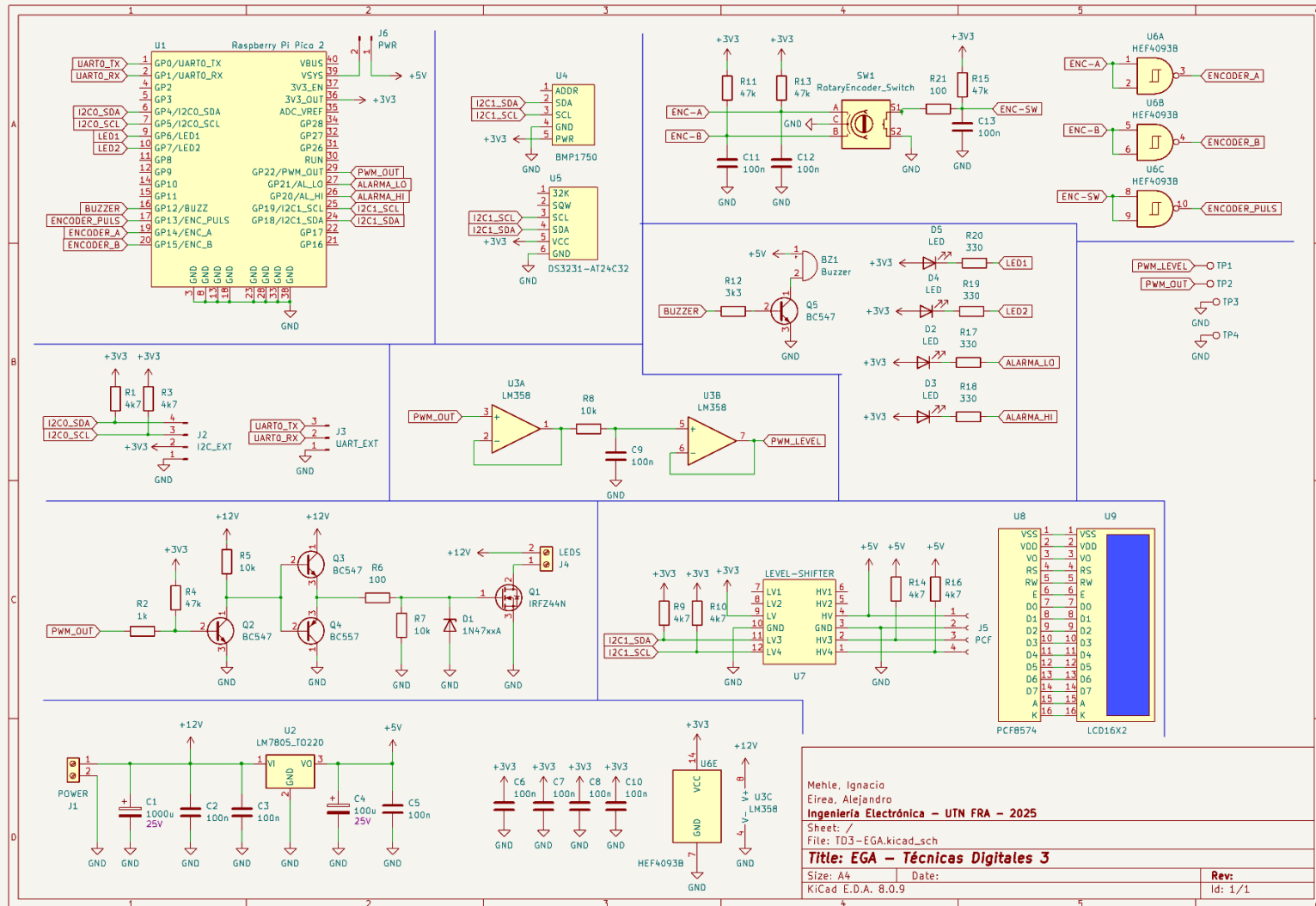


Fig. 13 Esquemático